Lecture 28.09: Decision Tree Classifier Implementation in Scikit-Learn

📌 Executive Summary

Hands-on implementation of DecisionTreeClassifier, parameter configuration, class imbalance handling, inspecting tree attributes, and visualizing decision structures.

🏛️ Theoretical Formulation & Mathematical Architecture

💻 Scikit-Learn DecisionTreeClassifier Architecture

Scikit-Learn's `DecisionTreeClassifier` is an optimized Cython implementation of the CART (Classification and Regression Trees) algorithm supporting both dense and sparse float32 arrays.

Mathematical Feature Importance Formulation:

The importance of feature j is calculated as the sum over all internal nodes t split on feature j, weighted by the fraction of samples reaching node t:

$$\boxed{\text{Imp}(j) = \sum_{t \in \text{Nodes:split}(t)=j} \left(\frac{N_t}{N} \Delta I(t) \right)}$$

The scores are then normalized to sum to 1.0:

$$\text{Normalized Imp}(j) = \frac{\text{Imp}(j)}{\sum_{k=1}^{d} \text{Imp}(k)}$$

💻 Vectorized Python Implementation

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text
```

```
# Fit sample classifier
clf = DecisionTreeClassifier(max_depth=3, random_state=42)
clf.fit(X_syn, y_syn)

# Introspect attributes
print(f"Total Nodes: {clf.tree_.node_count}")
print(f"Max Depth:   {clf.tree_.max_depth}")
print(f"Feature Importances: {clf.feature_importances_}")

# Text tree visualization
tree_rules = export_text(clf, feature_names=[f"Feature_{i}" for i in range(5)])
print("
--- Tree Architecture ---")
print(tree_rules[:300], "...")
```

🔬 Key Engineering Takeaways & Invariants

1. **Axis-Aligned Partitioning**: Decision trees recursively split feature space along orthogonal boundaries without requiring feature scaling.
2. **Impurity Optimization**: Split selection deterministically maximizes impurity reduction (Information Gain for classification, Variance Reduction for regression).
3. **Generalization Safeguards**: Regularization via pre-pruning (`max_depth` , `min_samples_leaf`) and post-pruning (`ccp_alpha`) prevents noisy leaf memorization.